



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23432 – Software Construction

(REGULATION 2023)

RAJALAKSHMI ENGINEERING COLLEGE

Thandalam, Chennai-602015

Name : K ZARAA JUHI SAAI

Register No : 240701044

Year / Branch / Section : II / CSE

Semester : IV

Academic Year : 2025 - 2026

INDEX

S.No	Date	Title
1.	07-02-2026	Jira Environment Setup
2.	14-02-2026	Jira Project Setup and User Story Management
3.	21-02-2026	Setting Up Epics, Tasks, and User Stories for Project Planning
4.	28-02-2026	Sprint Planning in Jira
5.	07-03-2026	Poker Estimation using Jira
6.	14-03-2026	Designing Class and Sequence Diagrams for Simple Asset Management System
7.	04-04-2026	Designing Architectural and ER Diagrams for Simple Asset Management System
8.	11-04-2026	Testing - Test Suites and Test Cases in Jira
9.	18-04-2026	Load Testing and CI/CD Pipeline Setup
10.	25-04-2026	GitHub: Project Structure & Naming Conventions

EXP NO: 01**JIRA ENVIRONMENT SETUP**

Aim: To set up and access the Jira environment by creating an Atlassian account and configuring a workspace for the Simple Asset Management System project.

Steps for Installation and Setup

1. Open a web browser and navigate to <https://www.atlassian.com/software/jira>.
2. Sign in with your Atlassian account. If you do not have one, click Sign Up and register a new account with a valid email address.
3. Once logged in, the Atlassian Home page will be displayed with links to Jira, Teams, Studio, and other apps.
4. Click on Jira from the left sidebar or from the "Your Apps" section to open the Jira workspace.
5. Click the "+ Create" button at the top of the page to begin creating a new project or issue in the workspace.
6. Select the Scrum project template, which is suitable for agile sprint-based management of the Asset Management System.
7. Enter the project name as "Pentanova" (or your designated project name), configure access settings, and click Create.
8. Verify that the workspace is accessible and the project dashboard loads successfully.

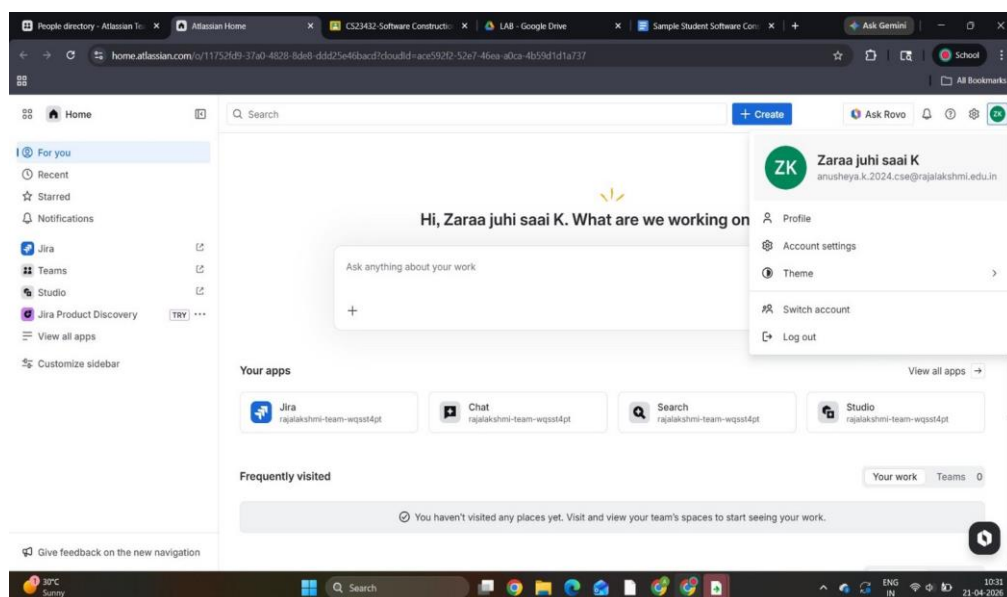


Figure 1: Atlassian Home Page – Jira Workspace for Simple Asset Management System

The screenshot above shows the Atlassian Home page after a successful login. The left sidebar provides access to Jira, Teams, and Studio. The "Your Apps" section displays the linked Jira, Chat, Search, and Studio apps, all connected to the rajalakshmi-team workspace. This confirms that the Jira environment has been set up and is accessible for project management.

Result: The Jira environment was successfully set up. A new Atlassian account and workspace were configured, and the Jira project home page for the Simple Asset Management System was accessed successfully.

EXP NO: 02**JIRA PROJECT SETUP AND USER STORY MANAGEMENT**

Aim: To create and configure a Jira project for the Simple Asset Management System and manage user stories for agile development.

1. Create a New Jira Project

9. Log in to the Jira workspace and click "Create Project" from the home dashboard.
10. Enter the project name as "Pentanova - Simple Asset Management System" and add a brief description.
11. Select the Scrum template for sprint-based planning and configure team access.
12. Click Create. The project dashboard will now display tabs: Summary, Backlog, Board, Calendar, Timeline, Pages, and Forms.

2. Navigate the Project Dashboard

13. Open the project and explore the Backlog tab, which is the primary area for managing all user stories, tasks, and epics.
14. The Board tab provides a Kanban-style view of tasks categorized as To Do, In Progress, and Done.

3. Create User Stories in the Backlog

15. From the Backlog, click "+ Create Issue" and select Story as the issue type.
16. Fill in user story details. Example: "As an admin, I want to add new assets so that all company resources are tracked in the system."
17. Create stories for all key features: Login, Registration, Profile, Approve Requests, Add Asset, Update Asset, Return Asset, Request Asset, View Asset, and ER Diagram.
18. Assign each story to an epic (e.g., User Management, Admin Management, Asset Management) and set priority and story point estimates.

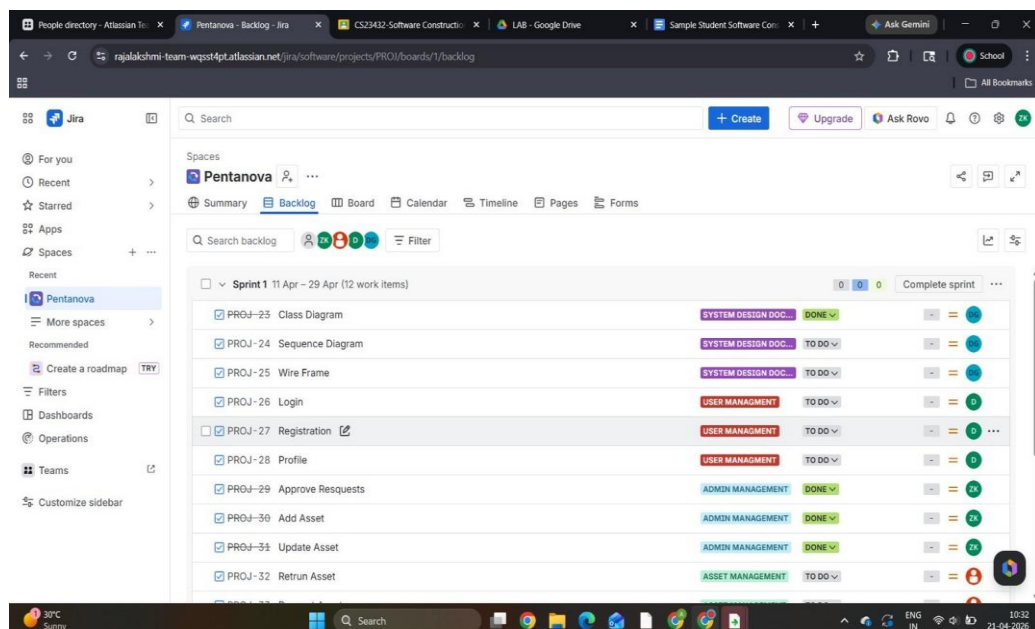


Figure 2: Jira Backlog – User Stories for Simple Asset Management System (Pentanova Project)

The backlog view above shows all user stories created for the project. Stories such as Class Diagram (PROJ-23), Sequence Diagram (PROJ-24), Wire Frame (PROJ-25), Login (PROJ-26), Registration (PROJ-27), Profile (PROJ-28), Approve Requests (PROJ-29), Add Asset (PROJ-30),

Update Asset (PROJ-31), and Return Asset (PROJ-32) are visible with their epic labels and status tags. This organized backlog forms the foundation for sprint planning.

4. View User Story Details

Each user story can be opened to view its full description, assignee, priority, parent epic, sprint, and story point estimate. For example, the "Add Asset" story (PROJ-30) describes the admin's need to add asset details such as asset name, ID, category, location, condition, and status to the database.

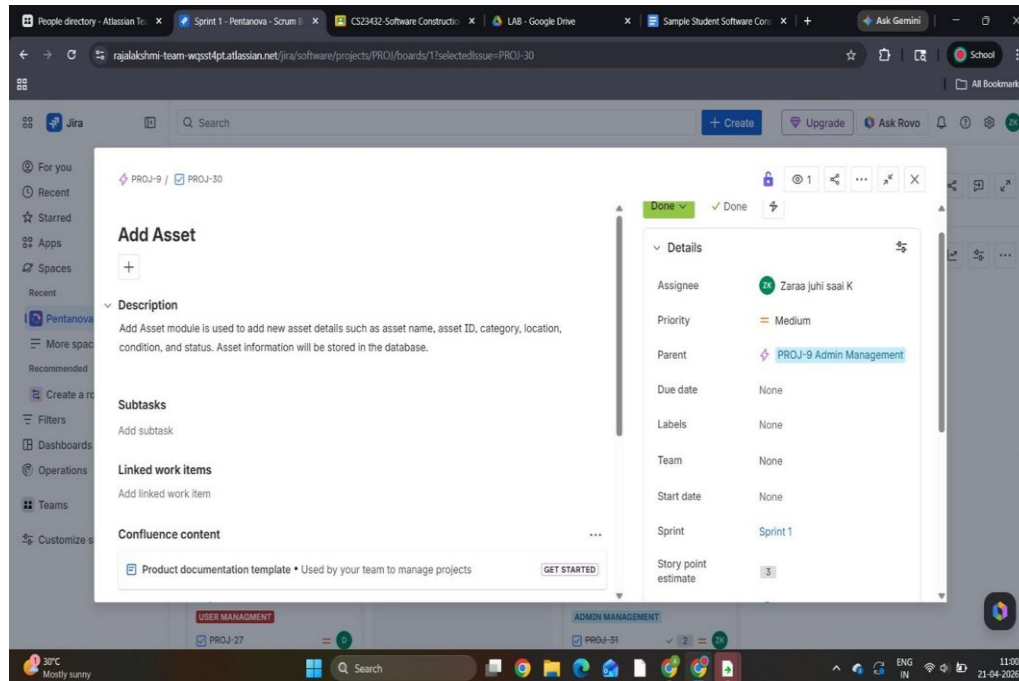


Figure 3: User Story Detail – Add Asset (PROJ-30) with Story Points and Epic Linkage

The screenshot shows the detailed view of the "Add Asset" user story (PROJ-30). It is assigned to "Zaraa juhi saai K", has a Medium priority, belongs to the PROJ-9 Admin Management epic, is placed in Sprint 1, and carries 3 story points. The description confirms the scope: adding asset name, ID, category, location, condition, and status to the database.

Result: A Jira project for the Simple Asset Management System was successfully created with all user stories added to the backlog, organized under appropriate epics, and detailed with descriptions and story point estimates.

EXP NO: 03**SETTING UP EPICS, TASKS, AND USER STORIES FOR PROJECT PLANNING**

Aim: To learn how to create epics, tasks, and user stories in Jira and organize them into a backlog for the Simple Asset Management System project.

1. Epics Created

Epics are large bodies of work that span multiple sprints. The following epics were created for the Simple Asset Management System:

- System Design Documentation - Covers all diagrams: Class Diagram, Sequence Diagram, Wire Frame.
- User Management - Handles employee login, registration, and profile management.
- Admin Management - Covers admin-level actions: Approve Requests, Add Asset, Update Asset.
- Asset Management - Covers employee-facing features: Return Asset, Request Asset, View Asset.
- Testing - Asset Management System - Covers all test suites and test cases.

2. User Stories Under Each Epic

Each epic was broken down into focused user stories describing specific user goals and acceptance criteria:

- PROJ-23: Class Diagram - Design the class diagram for the system (System Design Documentation).
- PROJ-24: Sequence Diagram - Design the sequence diagram for asset flow (System Design Documentation).
- PROJ-25: Wire Frame - Design the application wireframe (System Design Documentation).
- PROJ-26: Login - As a user, I want to log in securely to access the system (User Management).
- PROJ-27: Registration - As a new user, I want to register with my details (User Management).
- PROJ-28: Profile - As a user, I want to view and update my profile (User Management).
- PROJ-29: Approve Requests - As an admin, I want to approve asset requests (Admin Management).
- PROJ-30: Add Asset - As an admin, I want to add new assets to the system (Admin Management).
- PROJ-31: Update Asset - As an admin, I want to update existing asset details (Admin Management).
- PROJ-32: Return Asset - As an employee, I want to return an asset I no longer need (Asset Management).
- PROJ-33: Request Asset - As an employee, I want to request an asset for my work (Asset Management).
- PROJ-34: View Asset - As an employee, I want to view available assets and their status (Asset Management).

3. Sprint Board View

The Sprint Board provides a visual view of all tasks across To Do, In Progress, and Done columns. This helps the team track progress at a glance during the sprint.

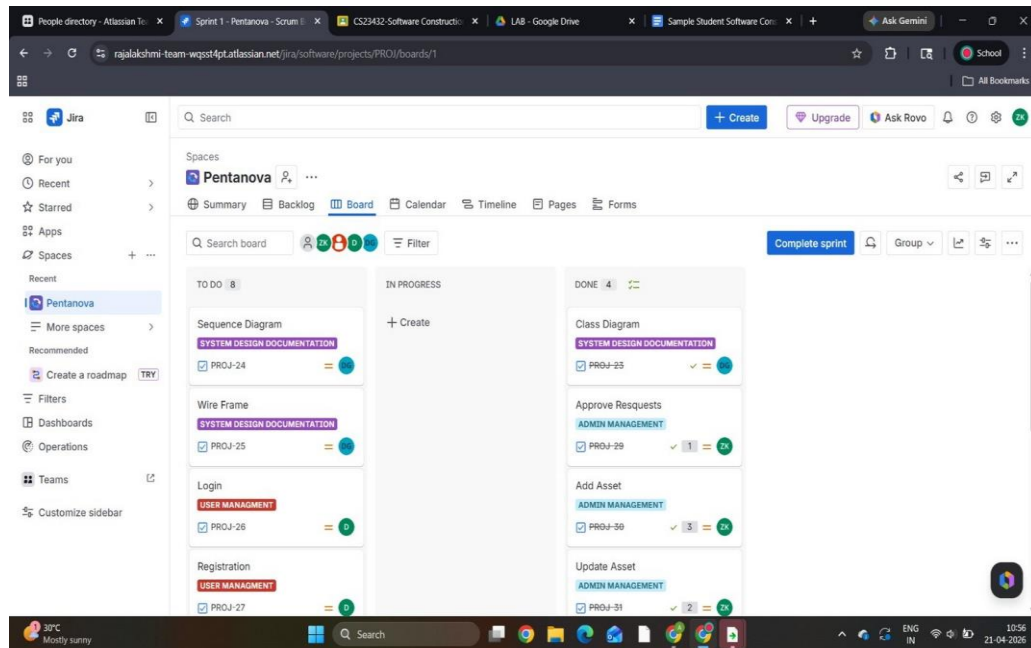


Figure 4: Jira Sprint Board – Kanban View of Sprint 1 (Simple Asset Management System)

The board above shows Sprint 1 in progress. The "To Do" column contains 8 items including Sequence Diagram, Wire Frame, Login, and Registration. The "Done" column shows completed items such as Class Diagram (PROJ-23), Approve Requests (PROJ-29), Add Asset (PROJ-30), and Update Asset (PROJ-31). This kanban view gives a real-time picture of sprint health and team velocity.

Result: Epics, user stories, and tasks were successfully created and organized in Jira for the Simple Asset Management System. The sprint board confirms stories are actively being worked on and completed as planned.

EXP NO: 04**SPRINT PLANNING IN JIRA**

Aim: To plan and assign user stories to specific sprints for the Simple Asset Management System project in Jira.

Sprint planning is the process of selecting user stories from the backlog and committing them to a time-boxed sprint, typically two weeks. Each sprint has a clear goal. For the Simple Asset Management System, the following sprint plan was executed:

Sprint 1 – System Design, Authentication, and Core Asset Operations (11 Apr – 29 Apr)

Sprint 1 was the primary sprint and included 12 work items across four epics:

- PROJ-23: Class Diagram (System Design Documentation) - DONE
- PROJ-24: Sequence Diagram (System Design Documentation) - To Do
- PROJ-25: Wire Frame (System Design Documentation) - To Do
- PROJ-26: Login (User Management) - To Do
- PROJ-27: Registration (User Management) - To Do
- PROJ-28: Profile (User Management) - To Do
- PROJ-29: Approve Requests (Admin Management) - DONE
- PROJ-30: Add Asset (Admin Management) - DONE
- PROJ-31: Update Asset (Admin Management) - DONE
- PROJ-32: Return Asset (Asset Management) - To Do
- PROJ-33: Request Asset (Asset Management) - To Do
- PROJ-34: View Asset (Asset Management) - To Do

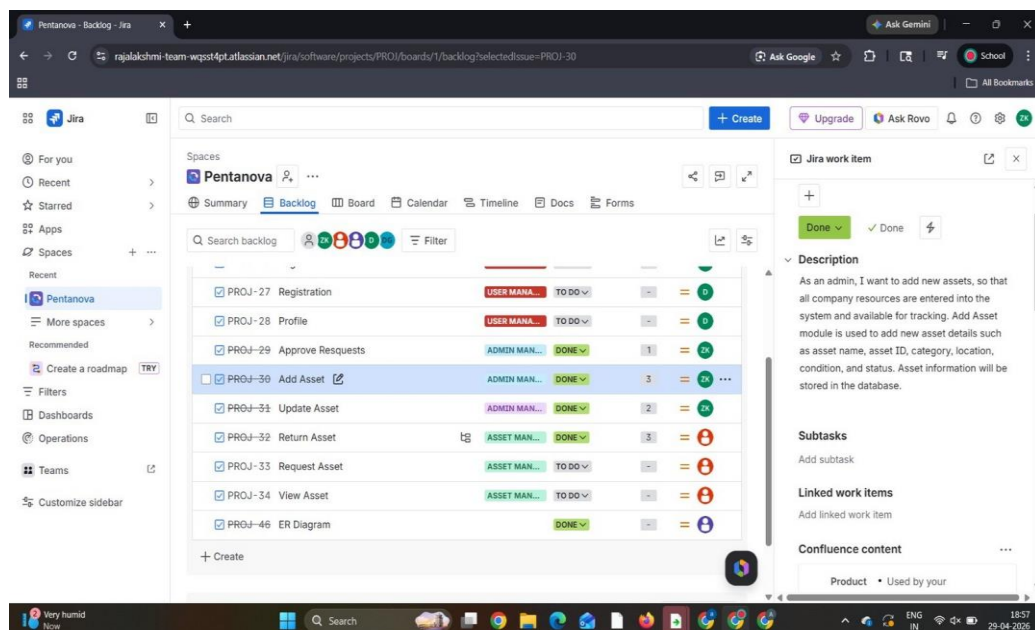


Figure 5: Sprint Planning – Backlog View Showing Sprint 1 Items and User Story Details

The screenshot above shows the Jira backlog with Sprint 1 active. Each user story is tagged with its epic label (e.g., Admin Management, Asset Management, User Management), a status badge (DONE or TO DO), story points, and an assignee. The right panel shows the description of the "Add Asset" story (PROJ-30), confirming it is assigned to Zaraa juhi saai K with 3 story points under the Admin Management epic.

Result: Sprint 1 was successfully planned and populated in Jira with 12 user stories for the Simple Asset Management System. Stories are distributed across system design, user management, admin operations, and asset management epics.

EXP NO: 05**POKER ESTIMATION USING JIRA**

Aim: To perform Poker Estimation for the user stories of the Simple Asset Management System using the planning poker technique and record story points in Jira.

Planning Poker is an agile consensus-based estimation technique. Team members assign story points to each user story based on complexity, effort, and uncertainty. Points follow the Fibonacci sequence: 1, 2, 3, 5, 8, 13, 21. Higher numbers indicate greater complexity.

Estimation Procedure

19. Open the Jira project backlog. Select a user story to estimate.
20. The Scrum Master reads the story aloud and clarifies acceptance criteria with the team.
21. Each team member independently selects a story point card without revealing it to others.
22. All cards are revealed simultaneously. Wide disagreements trigger a discussion round.
23. After discussion, the team re-votes until a consensus is reached. The agreed value is set as the story point estimate in Jira.
24. Repeat for all user stories in the backlog.

Story Point Estimates – Simple Asset Management System

User Story	Story Points	Priority
User Registration and Login	3	High
Add Asset (Admin)	3	High
Update Asset Details	2	Medium
Request Asset (Employee)	5	High
View Asset List	2	Medium
Return Asset	3	High
Approve / Reject Asset Request	1	High
Create Maintenance Request	5	Medium
ER and Architecture Diagrams	1	Low

The story points reflect the relative complexity of each feature. "Request Asset" and "Create Maintenance Request" received 5 points each due to their multi-step workflows involving approval, status updates, and notifications. "Add Asset" received 3 points as it involves form validation, API calls, and database writes. "Approve/Reject Asset Request" received only 1 point as it is primarily a status toggle with minimal logic.

Result: Poker Estimation was successfully completed for all user stories in the Simple Asset Management System project. Story points were recorded in Jira, enabling the team to plan sprint capacity and velocity effectively.

EXP NO: 06**DESIGNING CLASS AND SEQUENCE DIAGRAMS FOR SIMPLE ASSET MANAGEMENT SYSTEM**

Aim: To design a Class Diagram and Sequence Diagram that represent the structural model and behavioral flow of the Simple Asset Management System.

6A. Class Diagram

The Class Diagram defines the static structure of the system — the classes involved, their attributes, and the relationships between them. The following classes form the core model of the Simple Asset Management System:

- **Asset** - Represents a physical or digital asset with attributes: assetId, name, category, purchaseDate, status, and location.
- **Assignment** - Records which asset is assigned to which employee. Attributes: assignmentId, assignedDate, returnDate, conditionOnReturn.
- **Employee** - System users who request and use assets. Attributes: employeeId, fullName, department, email.
- **MaintenanceRecord** - Logs all maintenance activities for an asset. Attributes: maintenancId, serviceDate, vendor, cost, notes.
- **Vendor** - External service providers who handle maintenance. Attributes: vendorId, vendorName, contactEmail, phone.

Key relationships: An Asset has many Assignments and MaintenanceRecords. Each Assignment is linked to one Asset and one Employee. A MaintenanceRecord belongs to an Asset and is serviced by a Vendor.

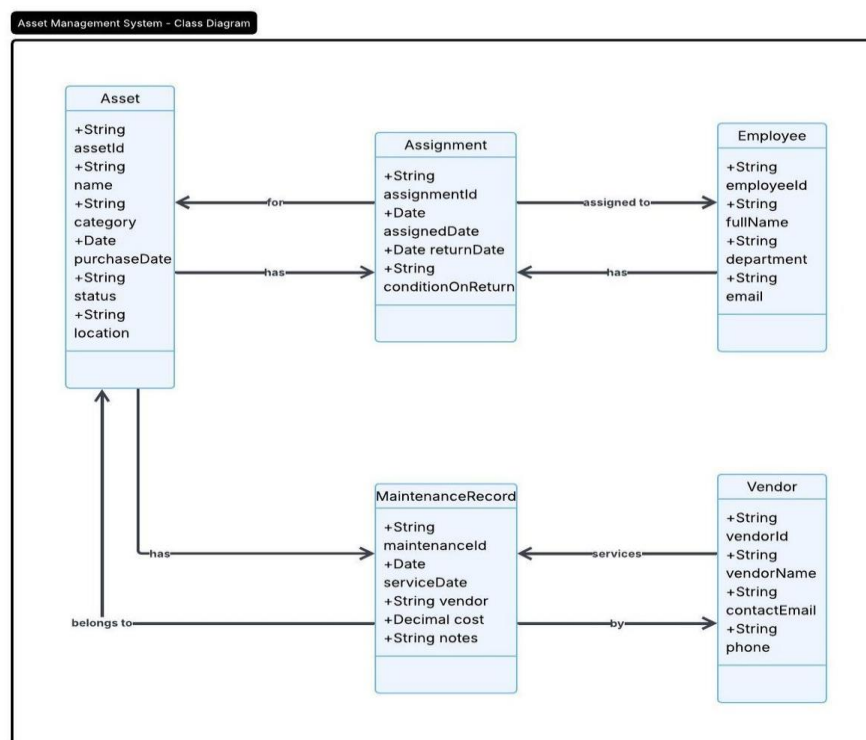


Figure 6: Class Diagram – Simple Asset Management System

The class diagram above shows the five main classes and their associations. The Asset class is at the center, connected to Assignment (via "has" and "for" relationships), MaintenanceRecord (via

"has" and "belongs to"), Employee (via Assignment), and Vendor (via MaintenanceRecord). This design ensures clear ownership of data and separation of concerns across system entities.

6B. Sequence Diagram

The Sequence Diagram models the time-ordered message flow between system actors and components for the core use case — Asset Request and Maintenance Handling. Participants: Employee, Asset Management System, Asset Database, Maintenance Service, and Vendor.

Interaction flow:

25. Employee sends login credentials. The system authenticates via the Asset Database and returns a success response.
26. Employee requests to view available assets. The system queries the database and displays the asset list.
27. Employee requests a specific asset by ID. The system checks asset status in the database.
28. [Asset Available]: The system assigns the asset to the employee and confirms the assignment.
29. [Asset Unavailable or Faulty]: The system notifies the employee and creates a maintenance request with the Maintenance Service.
30. The Maintenance Service contacts the Vendor to accept and schedule the service.
31. The system updates the asset status to "Under Maintenance" in the database and notifies the employee.

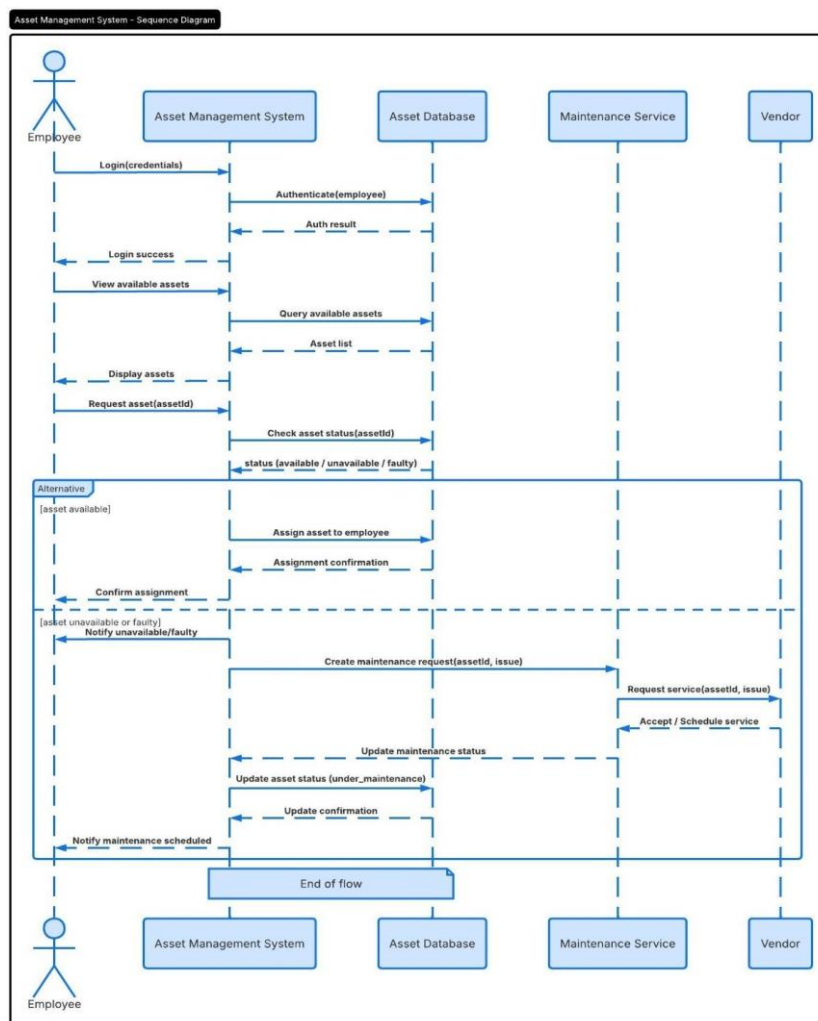


Figure 7: Sequence Diagram – Asset Request and Maintenance Flow

The sequence diagram clearly illustrates both the happy path (asset available → assign → confirm) and the alternate path (asset faulty → maintenance request → vendor scheduling → status update → notification). The "Alternative" frame separates these two branches, making the conditional logic easy to trace for developers and testers alike.

Result: The Class Diagram and Sequence Diagram for the Simple Asset Management System were designed and documented successfully. Both diagrams accurately reflect the system structure and the behavioral flow of the core asset management use case.

EXP NO: 07**DESIGNING ARCHITECTURAL AND ER DIAGRAMS FOR SIMPLE ASSET MANAGEMENT SYSTEM**

Aim: To design an Architectural Diagram and an Entity-Relationship (ER) Diagram for the Simple Asset Management System.

7A. Architectural Diagram

The system architecture follows a layered approach organized into five zones: Users, Frontend Layer, Application Layer, Data Layer, and supporting Security & Platform services. Each zone has clearly defined responsibilities:

- Users - Employees and Admins interact with the system through a Web App or Mobile App.
- Frontend Layer – Web App and Mobile App communicate with all backend services through the API Gateway / Load Balancer, which handles routing and load distribution.
- Admin & Operations - A Reporting and Analytics Module generates reports and dashboards for admins.
- Application Layer - The Asset Management Service handles all core business logic. The Authentication Service validates JWT tokens and enforces access control using IAM / RBAC Policies.
- Data Layer - Comprises the User/Auth Database (for credentials), Relational Database (asset records), File Storage (photos and documents), and Audit Log Store (event logs).
- Notification Service - Sends email and SMS alerts via an external Email/SMS Provider when assets are assigned, returned, or sent for maintenance.
- Integrations - A Barcode/QR Scanner allows employees and admins to scan asset tags for quick lookup and status updates.
- Backup & Recovery - Provides scheduled backup and restoration capabilities for all data stores.

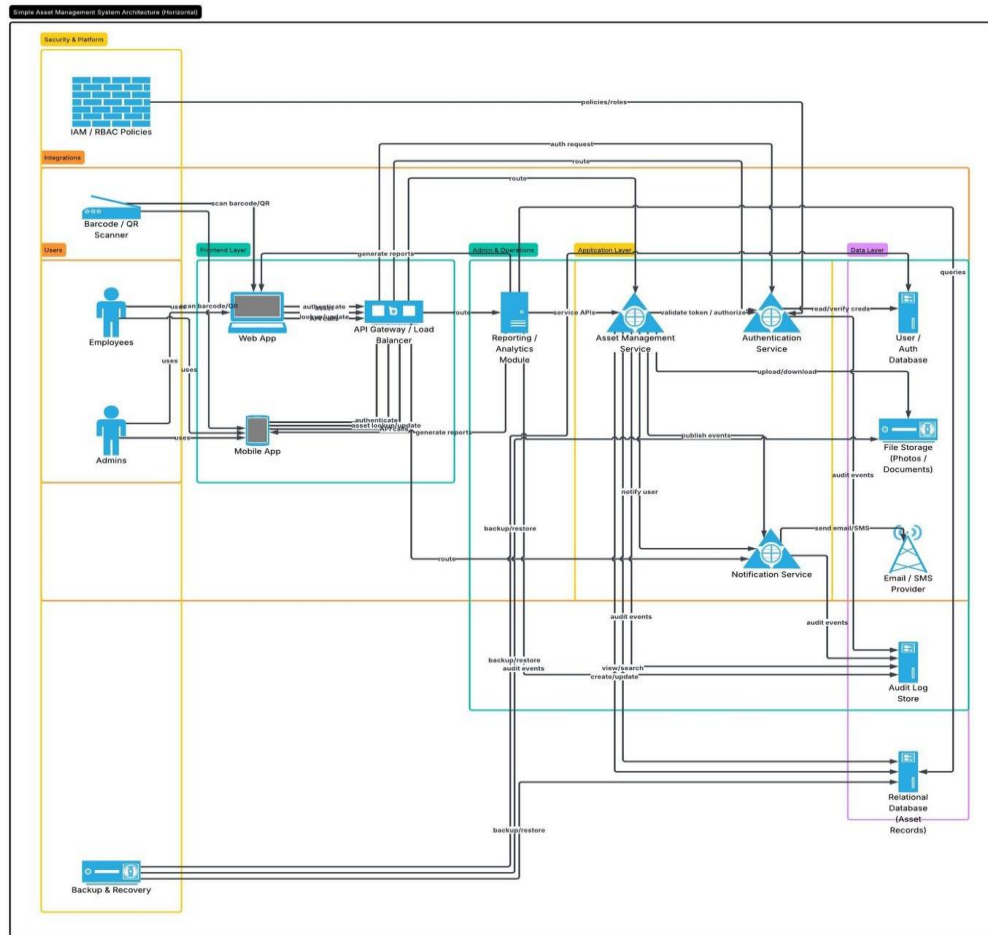


Figure 8: System Architecture Diagram – Simple Asset Management System

The horizontal architecture diagram above maps every component of the system and its interactions. Data flows from the user-facing apps through the API gateway to the application services and down to the data layer. Security is enforced at every layer through IAM/RBAC, and all significant events are captured in the Audit Log Store for compliance and traceability.

7B. ER Diagram

The ER Diagram defines the relational database schema underlying the Simple Asset Management System. The six main entities and their attributes are:

- **ASSET** (asset_id PK, asset_tag, asset_type_id FK, location_id FK, purchase_date, status, serial_number) – The central entity representing each tracked item.
- **ASSET_TYPE** (asset_type_id PK, name, category_code, description, default_value, depreciation_years, active) – Classifies assets by type.
- **LOCATION** (location_id PK, name, building, floor, room, city, state, country, is_active) – Stores the physical location of each asset.
- **ASSET_ASSIGNMENT** (assignment_id PK, asset_id FK, user_id FK, assigned_on, returned_on, assignment_status) – Tracks which user holds which asset.
- **USER** (user_id PK, first_name, last_name, email, department, created_at) – Represents employees who use the system.
- **MAINTENANCE_RECORD** (maintenance_id PK, asset_id FK, vendor_name, cost, service_type, status, service_date) – Records all maintenance events.

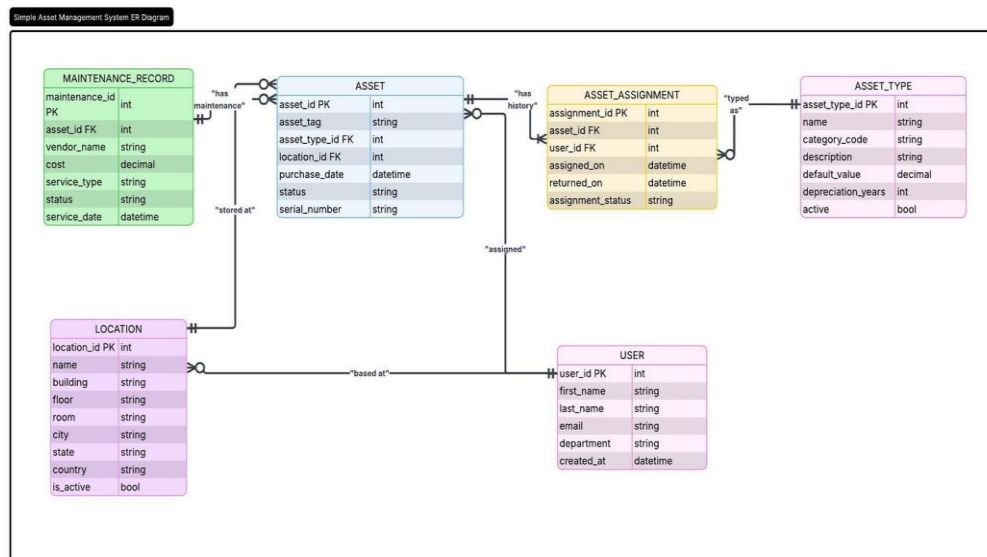


Figure 9: ER Diagram – Simple Asset Management System Database Schema

The ER diagram above shows the six database entities and the foreign-key relationships connecting them. ASSET is the central entity, linked to ASSET_TYPE ("typed as"), LOCATION ("stored at"), ASSET_ASSIGNMENT ("has history"), and MAINTENANCE_RECORD ("has maintenance"). USER connects through ASSET_ASSIGNMENT ("assigned") and is "based at" a LOCATION. This normalized schema supports all system operations efficiently.

Result: The Architectural Diagram and ER Diagram for the Simple Asset Management System were designed successfully. Both diagrams clearly capture the technical structure and database schema required for implementation.

EXP NO: 08**TESTING – TEST SUITES AND TEST CASES IN JIRA**

Aim: To create test plans and test cases in Jira, and write at least two test cases (happy path and error scenario) for five user stories of the Simple Asset Management System.

Test Case Design Procedure

- Identify core features to test: Login, Add Asset, View Asset, Update Asset, Delete/Return Asset.
- For each feature, define a happy path test case (normal expected behavior) and an error path test case (invalid input or edge case).
- Structure each test case with: TC ID, User Story, Test Steps, Input Data, Expected Result, Actual Result, and Pass/Fail status.
- Group related test cases into named Test Suites: TS01 Login Testing, TS02 Add Asset Testing, TS03 View Asset Testing, TS04 Update Asset Testing, TS05 Delete Asset Testing.
- Link each test suite to its parent epic in Jira and mark all as Done upon completion.

Test Suites – Testing Epic (PROJ-40)

The Testing – Asset Management System epic (PROJ-40) contains five test suites, each 100% complete:

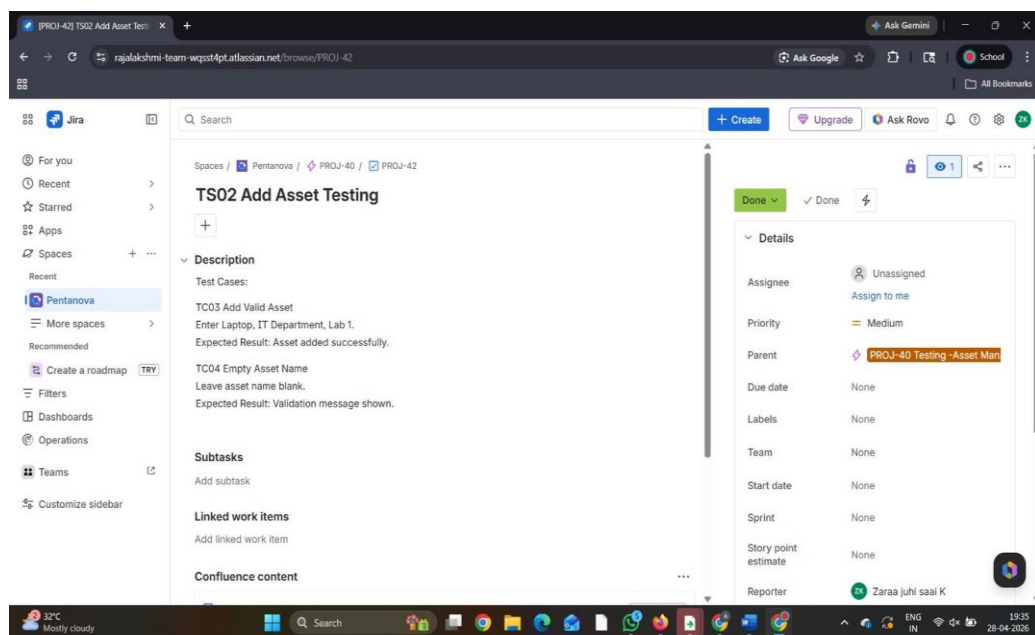


Figure 10: Jira Testing Epic (PROJ-40) – Test Suites with 100% Completion Status

The screenshot shows the "Testing – Asset Management System" epic (PROJ-40) with five child test suites: TS01 Login Testing, TS02 Add Asset Testing, TS03 View Asset Testing, TS04 Update Asset Testing, and TS05 Delete Asset Testing. All five suites are marked DONE with a 100% completion bar, confirming that all planned testing has been executed.

Test Suite Detail – TS02 Add Asset Testing (PROJ-42)

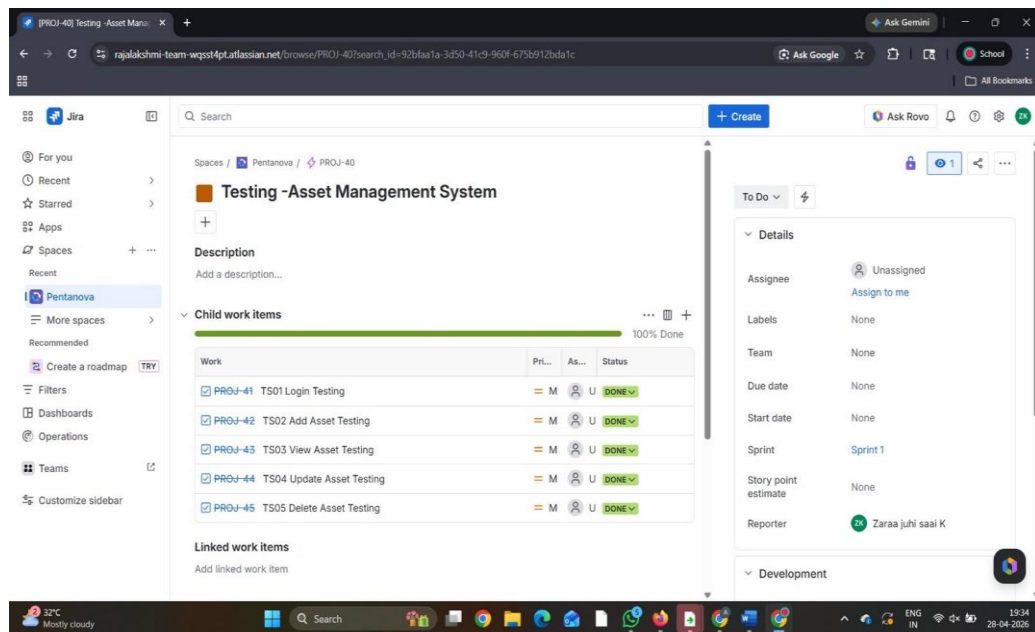


Figure 11: Jira Test Suite Detail – TS02 Add Asset Testing (PROJ-42)

The screenshot above shows the detail view of PROJ-42 (TS02 Add Asset Testing). It contains two test cases: TC03 Add Valid Asset (Enter Laptop, IT Department, Lab 1 – Expected: Asset added successfully) and TC04 Empty Asset Name (Leave asset name blank – Expected: Validation message shown). This confirms both the happy path and error path for the Add Asset feature were tested and documented in Jira.

User Stories Under Test

- US01 - As a user, I want to log in securely to access the system (TS01 Login Testing).
- US02 - As an admin, I want to add new assets to the system (TS02 Add Asset Testing).
- US03 - As a user, I want to view the list of available assets (TS03 View Asset Testing).
- US04 - As an admin, I want to update existing asset details (TS04 Update Asset Testing).
- US05 - As an admin, I want to delete or deactivate assets from the system (TS05 Delete Asset Testing).

Test Cases

TC01 & TC02 – US01: Login Testing

TC ID	User Story	Test Steps	Input	Expected Result	Actual Result	Status
TC01	US01 - Login	Enter valid credentials and click Login	Email: admin@rec.ac.in Pwd: Admin@123	Dashboard displayed	Dashboard displayed	Pass
TC02	US01 - Login	Enter invalid credentials and click Login	Email: wrong@mail.com Pwd: wrongpwd	Error: Invalid credentials	Error message shown	Pass

TC03 & TC04 – US02: Add Asset Testing

TC ID	User Story	Test Steps	Input	Expected Result	Actual Result	Status
TC03	US02 - Add	Fill all fields and	Laptop, IT	Asset added	Asset saved	Pass

	Asset	submit	Dept, Lab 1	successfully		
TC04	US02 – Add Asset	Submit with blank asset name	Name: (blank)	Validation error shown	Validation shown	Pass

TC05 & TC06 – US03: View Asset Testing

TC ID	User Story	Test Steps	Input	Expected Result	Actual Result	Status
TC05	US03 - View Asset	Navigate to Asset List page	No input	All assets displayed	Assets listed	Pass
TC06	US03 - View Asset	Search with invalid asset ID	ID: INVALID999	Error: Asset not found	Error shown	Pass

TC07 & TC08 – US04: Update Asset Testing

TC ID	User Story	Test Steps	Input	Expected Result	Actual Result	Status
TC07	US04 – Update Asset	Edit asset status and save	Asset A001: Active → Inactive	Status updated	Status changed	Pass
TC08	US04 – Update Asset	Submit update with empty required field	Location: (blank)	Validation error shown	Validation shown	Pass

TC09 & TC10 – US05: Delete Asset Testing

TC ID	User Story	Test Steps	Input	Expected Result	Actual Result	Status
TC09	US05 - Delete Asset	Select asset and click Delete/Deactivate	Asset A005	Asset removed from list	Asset removed	Pass
TC10	US05 - Delete Asset	Try to delete an assigned asset	Asset A001 (assigned)	Error: Cannot delete assigned asset	Error shown	Pass

Result: Test suites and test cases were successfully created in Jira for the Simple Asset Management System. Ten test cases covering five user stories were documented and executed, validating both happy path and error scenarios. All test suites are marked 100% Done.

EXP NO: 09**LOAD TESTING AND CI/CD PIPELINE SETUP**

Aim: To perform load testing on the Simple Asset Management System and set up a CI/CD pipeline using GitHub Actions to automate the build, test, and deployment process.

Part A: Load Testing with Apache JMeter

Load testing measures how the system performs under simulated concurrent user traffic. Apache JMeter was used to stress-test the asset management endpoints.

32. Download and install Apache JMeter from <https://jmeter.apache.org>.
33. Open JMeter and create a new Test Plan named "Asset Management Load Test".
34. Add a Thread Group with the following configuration: Number of Users (Threads): 100, Ramp-Up Time: 10 seconds, Loop Count: 5.
35. Add an HTTP Request sampler: Server Name: localhost (or deployed server IP), Port: 8080, Path: /api/assets.
36. Add the following Listeners to capture results: View Results Tree, Summary Report, Graph Results.
37. Save the test plan, click Start, and observe the following metrics: Response Time, Throughput, Error Percentage, CPU Usage.

Part B: CI/CD Pipeline with GitHub Actions

A CI/CD pipeline automates build, test, and packaging every time code is pushed to the main branch. This ensures the application is always in a deployable state.

38. Open the GitHub repository for the Simple Asset Management System.
39. Go to Settings > Integrations and link the repository to the Jira project (Pentanova).
40. In the repository, create the file: .github/workflows/main.yml
41. Configure the pipeline with the following stages: Java environment setup (JDK 17), Maven dependency installation, project compilation, unit test execution, and JAR packaging.
42. Commit and push the workflow file to trigger the pipeline automatically.
43. Monitor the Actions tab in GitHub to review build logs, test results, and deployment status.

main.yml – GitHub Actions Workflow

```
name: Java CI Pipeline
on:
  push:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4
      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
      - name: Build Project
        run: mvn clean compile
      - name: Run Tests
        run: mvn test
      - name: Package Application
        run: mvn package
      - name: Success Message
        run: echo "Asset Management System Build Successful!"
```

The pipeline includes: JDK 17 setup, Maven dependency installation, project compilation, unit test execution, JAR packaging, and a success message confirmation. Each push to the main branch triggers this workflow automatically, ensuring every code change is validated before it reaches production.

Result: Load testing was performed successfully on the Simple Asset Management System using JMeter. A CI/CD pipeline was also configured using GitHub Actions, which automatically builds, tests, and packages the application on every commit to the main branch.

EXP NO: 10**GITHUB: PROJECT STRUCTURE & NAMING CONVENTIONS**

Aim: To provide a clear and organized view of the project's folder structure and naming conventions for the Simple Asset Management System, helping contributors understand and navigate the codebase.

Repository Overview

The project follows a standard Spring Boot MVC architecture for the backend, with a modular frontend structure. The GitHub repository is organized as follows:

Folder / File	Purpose
README.md	Project overview, setup guide, and team details
.github/workflows/main.yml	GitHub Actions CI/CD pipeline configuration
src/main/java/...	Java source - controllers, services, repositories, models
src/main/resources/	application.properties - DB and server config
src/test/java/...	Unit and integration test classes
src/main/resources/static/	Frontend static files (HTML, CSS, JS)
src/main/resources/templates/	Thymeleaf HTML templates
docs/	ER diagram, architecture, wireframes, diagrams
pom.xml	Maven build config and dependency management
.gitignore	Files excluded from version control

Naming Conventions

- Java Classes - PascalCase: AssetController.java, MaintenanceService.java, UserRepository.java, AssetAssignment.java.
- HTML Templates - kebab-case: asset-list.html, add-asset.html, request-asset.html, maintenance-request.html.
- CSS / JS Files - kebab-case: asset-style.css, dashboard.js, request-form.js.
- Database Tables - snake_case: asset_assignment, maintenance_record, asset_type, location.
- REST API Endpoints - RESTful pattern: GET /api/assets, POST /api/assets, PUT /api/assets/{id}, DELETE /api/assets/{id}.
- Git Branches - Descriptive prefix: feature/add-asset, feature/request-asset, bugfix/login-issue, release/v1.0.

Application Wireframe – Navigation Flow

The wireframe below shows the complete navigation flow of the Simple Asset Management System, from the Login Screen through the Dashboard to all key functional pages.

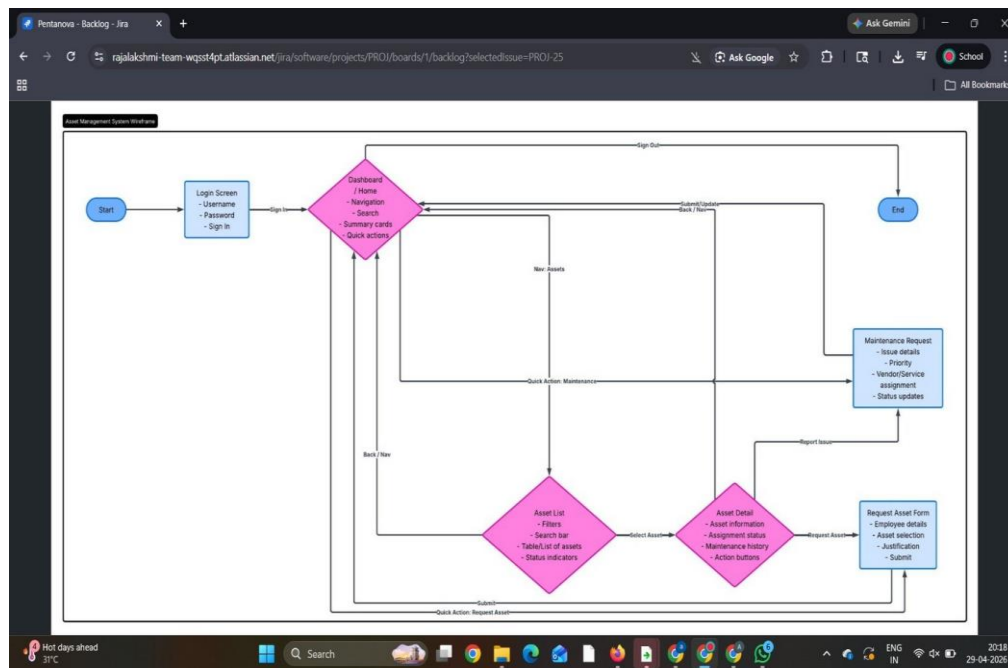


Figure 12: Application Wireframe – Navigation Flow (Simple Asset Management System)

The wireframe maps the full user journey. Starting from the Login Screen (with username, password, and sign-in fields), the user is taken to the Dashboard which serves as the central hub with navigation links, a search bar, summary cards, and quick action buttons. From the Dashboard, the user can navigate to the Asset List page (with filters, search, and status indicators), select any asset to view its Asset Detail page (showing assignment status, maintenance history, and action buttons), and then proceed to either submit a Request Asset Form (with employee details, asset selection, justification, and submit) or raise a Maintenance Request (with issue details, priority, vendor assignment, and status updates). The wireframe confirms a clean and intuitive user experience designed to minimize navigation steps.

Result: The GitHub repository structure and naming conventions for the Simple Asset Management System were documented clearly. The organized folder hierarchy, consistent naming patterns, and wireframe navigation flow ensure the project is maintainable, contributor-friendly, and ready for collaborative development.